STAGE 2 ARCHITECTURAL PLAN: ONE-SHOT INFERENCE

Context Grounding & Autoregressive Constraint Engineering via Structural Demonstrations

Project Domain:	AI/ML Telecoms & Data Engineering	**Core Target Model:**	CodeGen-350M-Multi (Active Venv)
Baseline Metric:	4.6% Exact Match (EM)	**Execution Script:**	run_oneshot.py (Root Directory)
Config State:	Greedy Decoding (Temp=0.0)	**Evaluation Scope:**	Full Validation Set (1,034 Core Queries)

1. Experimental Objectives & Theoretical Basis

The initial Stage 1 Zero-Shot execution established a structural baseline accuracy limit of 4.6% Exact Match. Diagnostic analysis showed that while the base model holds functional knowledge of global SQL syntax rules (achieving 61% precision on Easy projections), it suffers an intense failure cascade due to **Schema Unawareness**. Without active tracking of permissible table columns, it hallucinates generic table layouts, rendering Hard and Extra Hard performance levels at exactly 0.0%.

The objective of Stage 2 is to inject structural context without mutating model parameter weights. By combining our pre-calculated DDL schema representations with an isolated, domain-independent text-to-SQL translation example, we supply the model with an explicit format boundary map. This mitigates generation length inflation and anchors keyword matching.

2. Prompt Compilation Framework

To preserve our current execution architecture, the prompt builder interface utilizes standard multi-line SQL comment blocks (`/* ... */`). This cleanly splits training data parameters from production variables without breaking the sequence models. The generation layout is specified below:

```
/* Example Task Mapping */
/* Database Schema: */
CREATE TABLE instructor (
    id int,
    name text,
    department text
);
/* Question: Find the names of all instructors working in the Physics department. */
SQL: SELECT name FROM instructor WHERE department = 'Physics';

/* Target Task Mapping */
/* Database Schema: */
[Dynamic SQL DDL Data Block parsed via src/loader.py]

/* Question: [Target Question String extracted from dev.json] */
SQL: SELECT
```

> **Key Optimization - Autoregressive Magnetizing:** Because CodeGen operates via casual next-token prediction, ending the prompt context explicitly on the execution keywords `SQL: SELECT` serves as a

> completion constraint. It eliminates conversational preambles and forces immediate extraction of column conditions.

3. Project File Changes & Pipeline Integration

To maintain software development standards, all modifications are isolated to local workspace structures, preventing changes to baseline configurations:

- **`src/loader.py`:** Preserve the active `load_spider_schema_maps()` and `construct_zero_shot_prompt()` methods intact. Append the new `construct_one_shot_prompt()` method directly to the interface.
- **`run_oneshot.py`:** Refactor this file to act as our primary orchestrator. It will stream records through `construct_one_shot_prompt`, load the token weights onto the NVIDIA T4 GPU accelerator, and direct generation streams directly into `outputs/stage2_one_shot/stage2_predictions.json`.
- **`run_eval.py`:** Re-run the evaluation wrapper suite locally to index the new metrics, outputting results directly to a permanent tracking sheet: `outputs/stage2_one_shot/evaluation_report.txt`.

4. Evaluation Metrics & Comparison Strategy

Our performance tracking strategy requires a structured baseline audit. The results generated by this one-shot prompt architecture will be evaluated against our frozen Stage 1 matrix. Primary monitoring indices include:

1. **Keyword Precision Check:** Target a lift from the current 27.4% baseline by measuring compliance with injected schema boundaries.
2. **Where Clause Filtering:** Evaluate if seeing an example operator constraints structure helps improve the model's low 10.2% baseline efficiency index.
3. **Cross-Tier Structural Resilience:** Monitor if structural demonstration acts as an effective anchor for complex queries to break through the 0.0% accuracy cliff on Hard evaluation tiers.